Instalación y operación del laboratorio Oracle

Efrain Retana Segura. EIF402 Administración de Bases de Datos. Universidad Nacional. II Ciclo 2026. Versión del manual: 26 de octubre de 2026.

Arquitectura de la solución

La aplicación comprende HTML, CSS y JavaScript sin servicios externos de fuentes o analítica; una API Flask; y persistencia opcional en Oracle 21c XE mediante python-oracledb 3.4.1 en modo Thin. El navegador nunca recibe la contraseña Oracle. El modo de demostración almacena datos sintéticos en localStorage; el modo Oracle no reutiliza esos datos automáticamente.

La API usa un pool de hasta cuatro conexiones, llamadas con límite de tiempo, consultas de texto fijo y parámetros para enviar el contenido al paquete. La dependencia se fija para reproducibilidad; las actualizaciones deben probarse antes de adoptarse. Gunicorn es el servidor WSGI previsto para Linux en Azure. El servidor local Flask solo se utiliza para desarrollo.

Requisitos y comprobaciones previas

Se necesita Python 3.11 o 3.12, PowerShell, Oracle Database 21c XE operativo, SQL*Plus o SQL Developer, una PDB de laboratorio en modo READ WRITE y acceso de un DBA autorizado para la instalación. El usuario indicó la versión 21. El listener local fue consultado y anuncia el servicio `xepdb1` y el puerto TCP 1521. La conexión de RCONF_APP a XEPDB1 se confirmó después mediante pruebas locales; la disponibilidad futura debe comprobarse de nuevo.

La opción TCPS del listener en el puerto 5500 corresponde al servicio de administración HTTP observado. No debe confundirse con un listener SQL TCPS para el driver de esta aplicación. La protección TCPS para SQL requiere configuración independiente y validación del certificado.

Antes de instalar, confirmar que no se esté usando información real y que los nombres RCONF_OWNER, RCONF_APP, RCONF_REVIEW, RCONF_RUNTIME y RCONF_READONLY no existan. Los scripts son de primera instalación y se detienen ante errores de creación. No eliminan esquemas ni migran versiones previas. Si existe una instalación anterior, revisar sus objetos y respaldarla antes de planificar cambios.

Iniciar la demostración

Desde la carpeta del proyecto ejecutar:

```
.\start-local.ps1
```

Se crea un entorno `.venv`, se instalan las dependencias fijadas y se inicia la aplicación en `http://127.0.0.1:8000`. El valor inicial es APP_MODE=demo. Para detenerla utilizar Ctrl+C. Si PowerShell bloquea el script, ejecutar manualmente los comandos `py -3 -m venv .venv`, `.venv\Scripts\python.exe -m pip install -r requirements.txt` y `.venv\Scripts\python.exe -m app.server`; no es necesario cambiar la política global de ejecución.

La demostración permite editar riesgos, avanzar fases, registrar referencias y exportar JSON. Los cambios solo pertenecen al perfil de ese navegador. Restablecer demostración borra esos cambios. Exportar antes de hacerlo si se desea conservar el ejercicio. No cargar secretos ni datos personales en la demostración.

Orden de instalación en Oracle

1. Abrir SQL Developer o SQL*Plus y conectar como DBA a la base de laboratorio. Ejecutar `sql/00_preflight.sql`. Revisar versión, contenedor y servicios. El nombre de instancia XE y el servicio de PDB XEPDB1 no son equivalentes.
2. Conectar directamente al servicio de PDB, por ejemplo `localhost:1521/XEPDB1`, con cuenta DBA autorizada. No ejecutar la creación en CDB$ROOT. El script 01 comprueba que no se esté en la raíz ni en PDB$SEED.
3. Ejecutar `sql/01_accounts.sql` con Run Script o F5 en SQL Developer. Introducir tres contraseñas diferentes cuando el script las solicite; usar valores aleatorios de 20 a 64 caracteres de letras, números y guion bajo. Esta limitación evita ambigüedades del mecanismo de sustitución de SQL*Plus. No registrar la ejecución con spool.

4. Conectar como RCONF_OWNER a la misma PDB y ejecutar `sql/02_schema.sql`. Deben compilarse el paquete y su cuerpo sin objetos inválidos. Si aparece un error de compilación, revisar SHOW ERRORS y detenerse antes de conceder uso operativo.
5. Opcional: como propietario ejecutar `sql/02b_seed_demo.sql` para cargar escenarios sintéticos. Se identifican como demostración en sus referencias. No cargarlo si se desea iniciar un registro vacío.
6. Como DBA o AUDIT_ADMIN en la misma PDB ejecutar `sql/03_audit.sql`. Configura una política del proyecto y no modifica el modo global de auditoría ni reinicia Oracle. Verificar la política según el estado de auditoría de la instalación.
7. Como RCONF_APP ejecutar `sql/05_verify_runtime.sql`. Como DBA ejecutar `sql/04_verify_dba.sql`, después de generar acciones permitidas y denegadas. Conservar los resultados en una ubicación restringida.
8. Como DBA ejecutar `sql/06_lock_owner.sql` al terminar esquema y carga opcional. Revoca permisos DDL de instalación y bloquea el inicio de sesión del dueño. El paquete con derechos del definidor continúa atendiendo a la aplicación.

En SQL*Plus un ejemplo de conexión sin exponer contraseña en el comando es:

```
sqlplus /nolog

CONNECT sys@localhost:1521/XEPDB1 AS SYSDBA

-- Introducir la contraseña solo en el prompt de SQL*Plus

@sql/01_accounts.sql
```

El acceso administrativo debe estar autorizado. No incluir contraseñas en URLs, capturas, archivos del repositorio ni comandos guardados en el historial.

Objetos y modelo de mínimo privilegio

RCONF_OWNER contiene RISKS, RISK_EVENTS, vistas, paquete y datos sintéticos de prueba. Su capacidad de crear objetos se utiliza durante instalación y se revoca al finalizar. El permiso temporal CREATE SEQUENCE es necesario porque las tablas usan columnas identity. No se usa como cuenta de ejecución web.

RCONF_APP recibe CREATE SESSION y el rol RCONF_RUNTIME. El rol permite SELECT sobre RISK_VIEW, EVENT_VIEW y MASKED_PERSONAL y EXECUTE sobre RISK_API. No recibe permisos sobre las tablas base, privilegios ANY, DBA, AUDIT_ADMIN ni concesiones administrativas.

RCONF_REVIEW recibe CREATE SESSION y RCONF_READONLY para consultar las mismas vistas, sin ejecutar el paquete. Es una cuenta de revisión de base de datos distinta del rol auditor de la web. En esta versión todos los roles web usan la misma cuenta técnica RCONF_APP, por lo que la separación entre analista, auditor y gestor se aplica en la API.

RISKS almacena el documento JSON y una versión para control de concurrencia. RISK_EVENTS registra el estado anterior y nuevo para cada cambio, junto con fecha y rol. La vista de eventos no publica esos documentos completos. La bitácora depende de las restricciones Oracle; un DBA conserva capacidad de alterar datos y por eso no debe describirse como evidencia inmutable.

RISK_API usa AUTHID DEFINER para encapsular escrituras. Valida que el rol declarado tenga escritura y que las decisiones finales correspondan a gestor. La API Flask realiza la validación completa de estructura y fases. La confianza termina en el servicio: quien obtenga las credenciales técnicas podría invocar el paquete declarando un rol. La prevención exige proteger esas credenciales; un diseño de mayor aseguramiento usaría identidades y contextos de seguridad individuales.

MASKED_PERSONAL presenta un identificador, una etiqueta genérica y un correo oculto de un conjunto sintético. Es una demostración de reducción de exposición mediante vista. No utiliza Data Redaction, no demuestra anonimización y no sustituye una política de tratamiento de datos personales.

Configurar el modo Oracle

```
Copy-Item .env.example .env
```

Editar `.env` localmente. Establecer APP_MODE=oracle, ORACLE_USER=RCONF_APP, la contraseña correspondiente y ORACLE_DSN=localhost:1521/XEPDB1. Configurar API_TOKEN_ANALISTA, API_TOKEN_GESTOR y API_TOKEN_AUDITOR con tres cadenas aleatorias distintas de al menos 32 caracteres. Se pueden generar localmente con:

```
.venv\Scripts\python.exe -c "import secrets; print(secrets.token_urlsafe(32))"
```

Generar un valor por rol, distribuirlo por un canal restringido y no incorporarlo a evidencia o repositorio. El `.gitignore` excluye `.env` y los directorios de evidencia local. Esto no protege un secreto ya publicado: en ese caso debe rotarse y revisarse su exposición.

Reiniciar el servidor, abrir el sitio y seleccionar Conectar sesión. El token se guarda solo en memoria JavaScript; recargar o cerrar sesión lo descarta. No se conserva en localStorage ni en la URL. La cuenta Oracle y su contraseña nunca se ingresan en el formulario web.

Para TCPS, configurar un DSN adecuado y la ruta del wallet usando ORACLE_WALLET_DIR y su contraseña cuando corresponda. El driver mantiene validación de identidad del servidor. Configurar el listener y certificados con la documentación Oracle; el proyecto no fabrica ni instala certificados ni presupone soporte de cifrado nativo NNE en modo Thin [S10].

Operación del sitio

El analista crea registros en Planificar y estado Abierto. Registra activo, responsable, fecha, valoración inherente, residual estimado, controles y tratamiento. Avanza una fase a la vez y agrega una referencia de evidencia. Las correcciones pueden regresar una fase o volver a Planificar.

El auditor consulta riesgos y eventos. El gestor puede crear y editar, además de registrar la decisión residual en Actuar con evidencia y justificación. Para Mitigado el producto residual debe ser como máximo 4. Una aceptación superior requiere justificación explícita; no se confunde con un control eficaz.

La vista PHVA ofrece una recomendación por riesgo y muestra el tratamiento registrado, los controles seleccionados, la evidencia esperada y el residual estimado. Estas sugerencias ayudan a preparar el siguiente paso; el responsable revisa la pertinencia de cada medida y el auditor contrasta los resultados con los archivos originales. La vista Controles permite filtrar por categoría y por riesgo del registro.

Al guardar se envía la versión leída. Si otra sesión modificó el registro, el servidor rechaza el cambio y solicita actualizar. El paquete bloquea el registro y comprueba nuevamente la versión antes de actualizar, para evitar perder cambios concurrentes. El riesgo y su evento se confirman juntos; no hay un endpoint de borrado de bitácora.

Comprobación guiada desde la aplicación local

Con APP_MODE=oracle y una sesión web conectada, abrir **Ciclo PHVA → Ejecutar pruebas de acceso mínimo**. El servidor utiliza exclusivamente la cuenta técnica RCONF_APP configurada en `.env`; el navegador no recibe ni envía su contraseña. La comprobación consta de cuatro consultas SELECT fijas: lectura de la vista MASKED_PERSONAL y verificación del valor `[oculto]`, denegación de la tabla DEMO_PERSONAL, lectura de RISK_VIEW y denegación de la tabla RISKS. Los intentos de tabla base usan `WHERE 1=0`, de modo que prueban el permiso sin extraer filas. Ningún SQL escrito por el usuario llega a este endpoint.

El resultado muestra conforme, fallo o revisar por prueba, cuenta, PDB y hora UTC. El botón **Descargar resultado JSON** conserva una copia técnica; antes de usarla como evidencia debe agregarse ejecutor, revisor, contexto, versión del proyecto y archivo original de la prueba. Una prueba permitida sobre la tabla base marca fallo. Un error distinto de ORA-00942 u ORA-01031 requiere investigación.

La ruta POST `/api/checks/runtime` requiere token, conexión desde el propio equipo y ausencia de WEBSITE_HOSTNAME; queda inhabilitada en Azure. No concede privilegios y no ejecuta DDL, UPDATE ni consultas DBA. Para la prueba negativa de UPDATE usar `sql/05_verify_runtime.sql` con RCONF_APP en SQL*Plus; para correlacionar la política y el rastro unificado usar `sql/04_verify_dba.sql` con DBA. No repetir los scripts 01–03 en una instalación existente. La comprobación local de cuatro SELECT ejecutada el 26 de octubre de 2026 quedó en `docs/09_comprobacion_local_2026-10-26.json`, con procedencia y límites explícitos.

Superficie de API y restricciones

GET /health indica que el proceso responde y comunica el modo; no comprueba Oracle. GET /api/config publica solo el modo. GET /api/session identifica el rol del token. GET /api/risks y GET /api/events requieren credencial en modo Oracle. POST /api/risks y PUT /api/risks/id requieren rol con escritura. POST /api/checks/runtime ejecuta únicamente las cuatro comprobaciones locales predefinidas; cualquier rol autenticado puede observarlas, sin permiso para ejecutar SQL arbitrario.

Se limita la carga a 16 KiB, se valida el tipo JSON y los campos tienen longitud máxima. Las respuestas de error ocultan SQL, contraseñas y detalles internos. Se usa una política CSP restringida al propio origen, no hay CORS abierto y los tokens se envían en Authorization. Deben revisarse el navegador y su entorno contra extensiones maliciosas y XSS; el escape de contenido es necesario pero no reemplaza las revisiones de seguridad.

Problemas frecuentes

ORA-12514: comprobar el nombre del servicio y que la PDB esté abierta. ORA-01017: revisar cuenta, contraseña y PDB; no aumentar privilegios para resolver una autenticación fallida. ORA-00942 durante una prueba negativa puede ser el resultado esperado; en una consulta positiva indica un objeto o permiso que debe revisarse.

Una respuesta HTTP 401 exige revisar el token del rol, no la contraseña Oracle del navegador. Una respuesta 403 expresa una restricción de función. Una respuesta 409 al guardar indica conflicto o intento de usar la API de escritura en demostración. Una respuesta 503 requiere revisar conectividad, objetos y configuración desde el servidor; el mensaje público no revela el detalle interno.

Si no aparecen eventos unificados, comprobar política, cuenta, contenedor, acciones generadas y posible demora de visibilidad del registro. La revisión se hace con privilegios de auditoría separados; no se concede acceso administrativo a RCONF_APP como solución.

Reversión y mantenimiento

Para volver a demostración cambiar APP_MODE=demo y reiniciar; se conservan los registros Oracle. Para deshabilitar solo la política del laboratorio revisar y ejecutar `sql/07_disable_audit.sql` con AUDIT_ADMIN. Esto no elimina cuentas, datos ni evidencias anteriores.

No se incluye un DROP USER CASCADE automático. La eliminación exige revisar dependencias y retención de evidencias con el DBA. Para mantenimiento, el DBA desbloquea temporalmente al propietario y concede únicamente los privilegios DDL necesarios; al terminar los revoca y vuelve a bloquear. Registrar el motivo y comparar permisos antes y después.

Fuentes técnicas: S8 a S12 en `06_fuentes.md`. El estudiante compartió las salidas de instalación y de `05_verify_runtime.sql` y `04_verify_dba.sql`; T11 requiere correlación. La comprobación local adicional de cuatro SELECT fue ejecutada por Codex en el entorno del estudiante y debe revisarse antes de incluirla en la presentación. TCPS y Azure siguen sin prueba de integración.

Referencias del documento

S8. Oracle. Database Security Guide, Configuring Privilege and Role Authorization. Fundamento de roles y permisos de objetos. https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/configuring-privilege-and-role-authorization.html

S9. Oracle. SQL Language Reference, CREATE AUDIT POLICY Unified Auditing. Referencia de sintaxis; verificar ejecución en 21c XE. https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/CREATE-AUDIT-POLICY-Unified-Auditing.html

S10. Oracle python-oracledb. Connecting to Oracle Database. Conexiones y pools. La documentación latest es cambiante; el proyecto fija 3.4.1. https://python-oracledb.readthedocs.io/en/latest/user_guide/connection_handling.html

S11. Microsoft Learn. Quickstart Deploy a Python web app to Azure App Service. https://learn.microsoft.com/en-us/azure/app-service/quickstart-python

S12. Microsoft Learn. Hybrid Connections in Azure App Service. TCP, Relay y agente local. https://learn.microsoft.com/en-us/azure/app-service/app-service-hybrid-connections